אוניברסיטת
אריאל
בשומרון

Slide set # 8
Object-Oriented Programming (1)

Introduction to Programming for Engineers

Dr. Inon Zuckerman, Dr. Chen Hajaj

Industrial Engineering and Management

חלק מהשקפים נלקחו מקורס דומה באוניברסיטת תל אביב ולהם התודה.

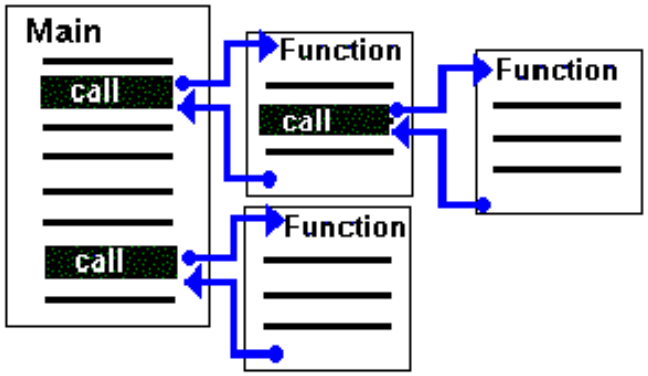
Object Oriented Programming

אוניברסיטת אריאל בשומרון

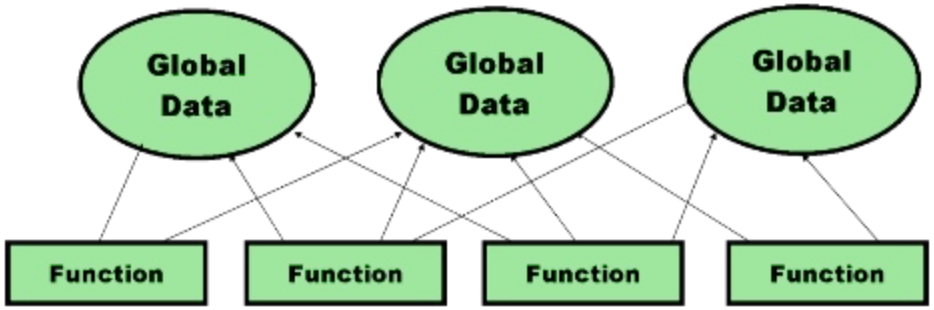
"Another trick in software is to avoid rewriting the software by using a piece that's already been written, so called component approach which the latest term for this in the most advanced form is what's called Object Oriented Programming" — Bill Gates.

Procedural Programming

- Up until now, we only used ***functions*** to organize our code into modular code blocks.
  - Why is this important ?

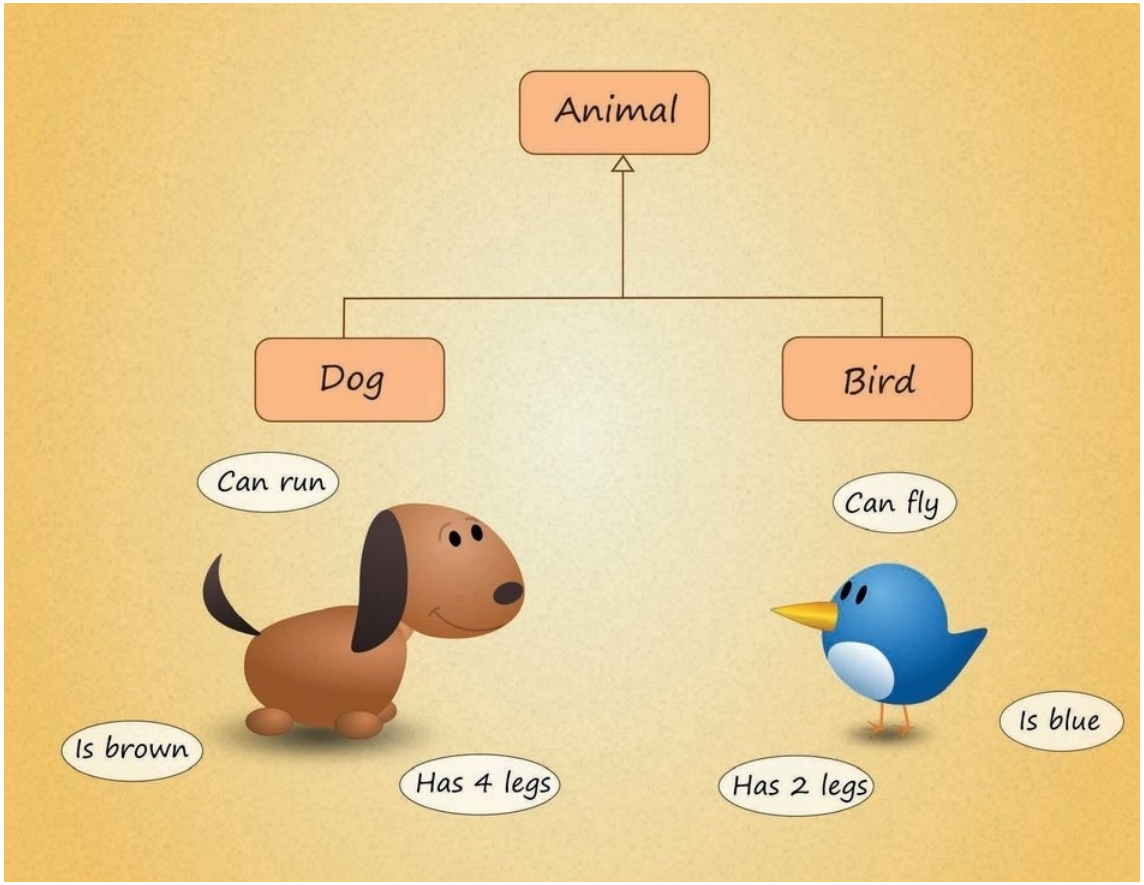


- How would we implement a software system for managing students' grades that way?
  - By this approach, data is usually being stored in global data structures and accessed by many different functions.



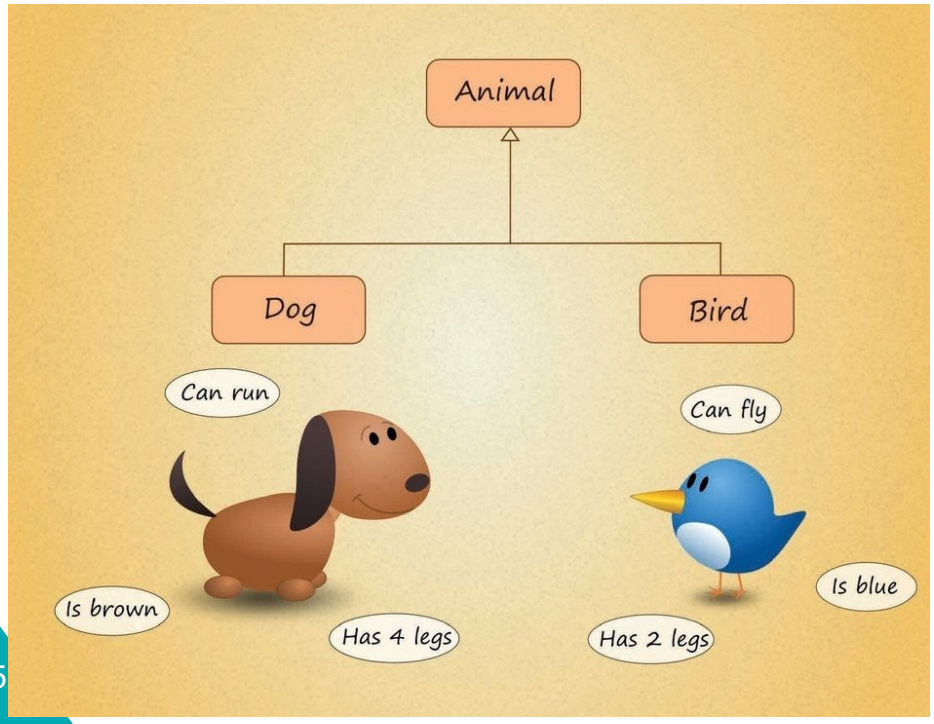
Object-Oriented Programming

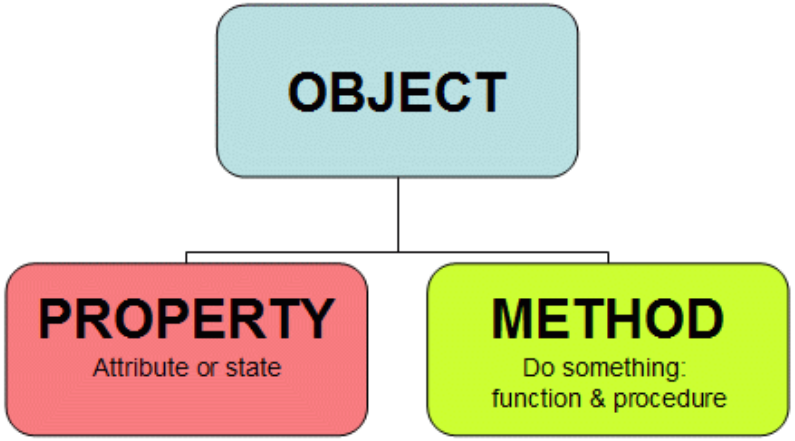
- In Object Oriented Programming (OOP), we model the entities of the real-world using **objects**.



Object-Oriented Programming

- In Object Oriented Programming, we model the entities of the real-world using **objects**.
- Objects holds both **code** (functions) and **data** for the entities they represent.



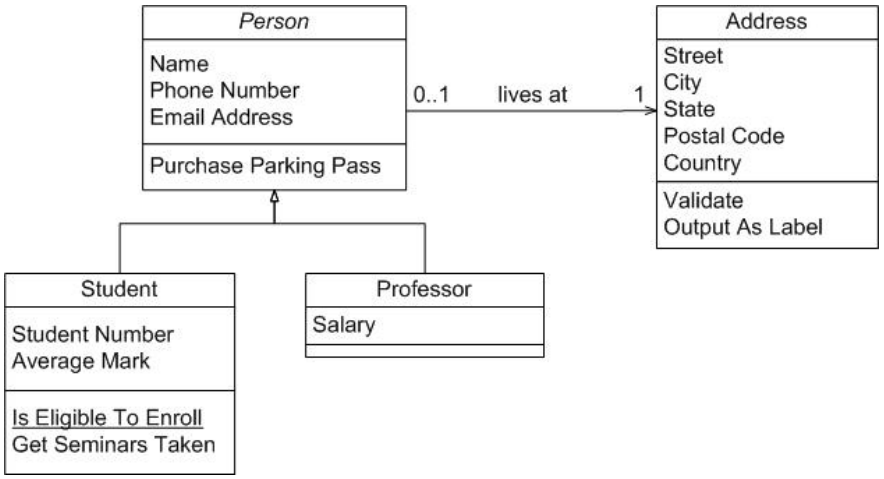


Object-Oriented Programming

אוניברסיטת אריאל בשומרון

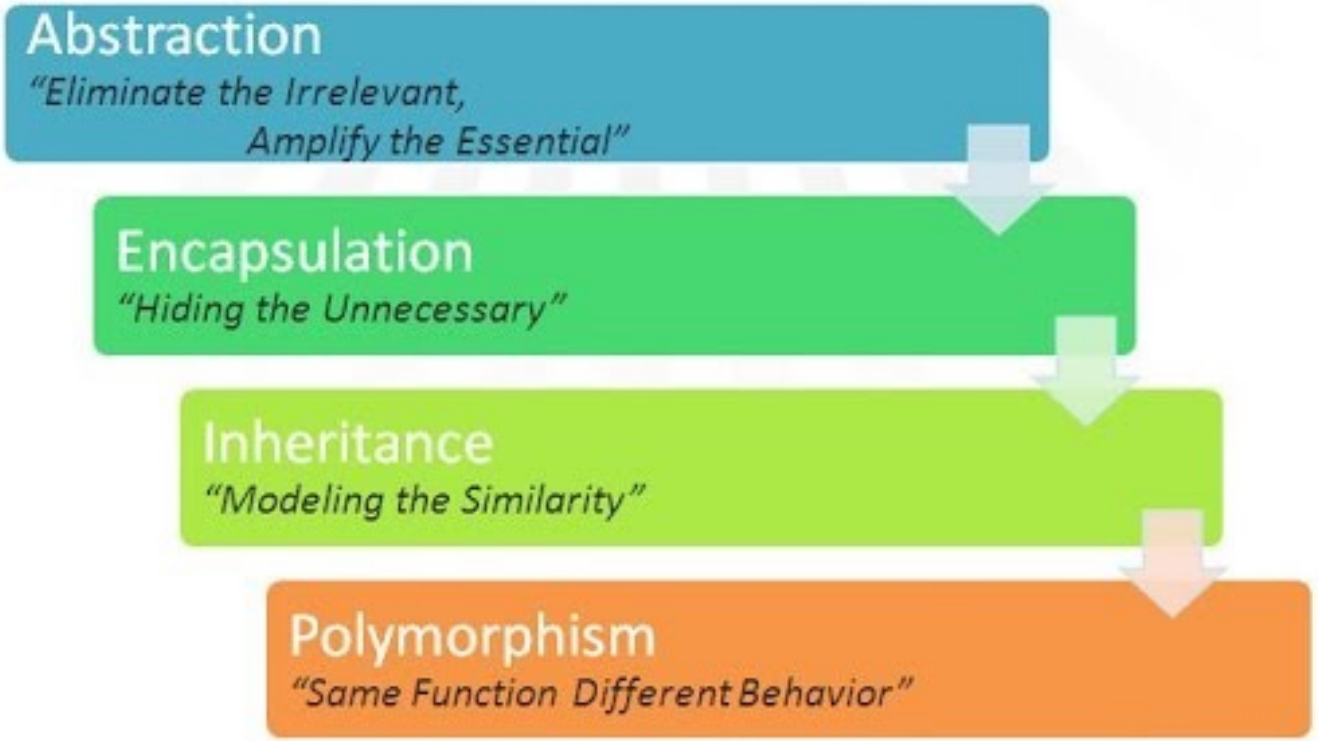
Wikipedia:

An **object-oriented** program may be viewed as a **collection of interacting objects**, as opposed to the conventional model, in which a program is seen as a list of tasks (subroutines) to perform.



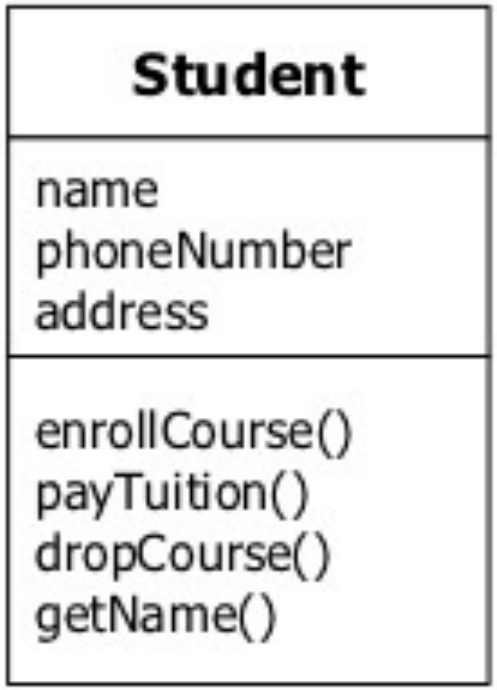
Characteristics of the Object-Oriented Paradigm

אוניברסיטת
אריאל
בשומרון



Example: An object that represents a Student

אוניברסיטת אריאל בשומרון

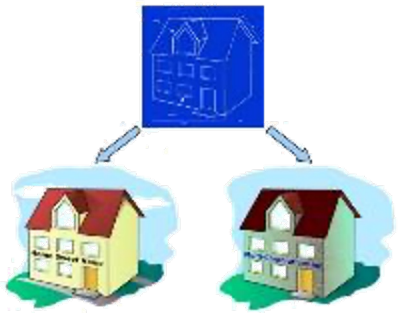


Data: Attributes (==Variables/Fields)

Functionality: Methods (==Functions)

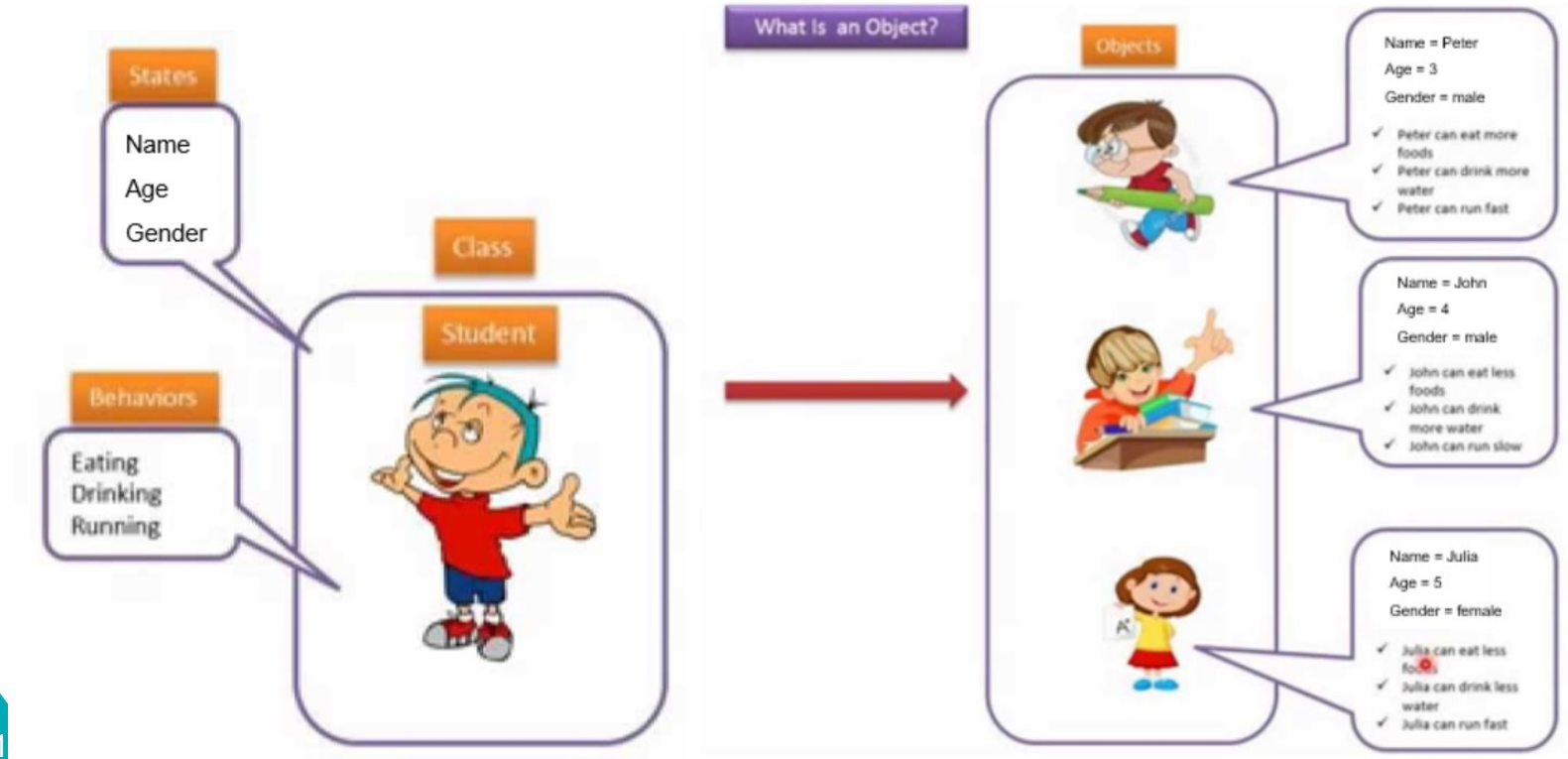
Classes are user-defined types

- In OOP, a **Class** is a piece of code that defines a new object **type**.
  - A class defines which **attributes (fields)** will be allocated in memory for each object (instance) of the class.
  - A class contains the code for the **functions** of a given object type.
- Classes can be viewed as a blueprint for creating objects.



Classes and Objects

- A class defines a new type of which many objects (instances) can be created.
  - Each object may hold different values for the class fields.



We already met some classes

אוניברסיטת אריאל בשומרון

```
In [1]: number = 2
        text = 'Sample text'

In [2]: print(type(number))
        print(type(text))

        <class 'int'>
        <class 'str'>
```

```
In [4]: dir(text)

Out[4]: ['__add__',
         '__class__',
         '__contains__',
         '__delattr__',
         '__dir__',
         '__doc__',
         '__eq__',
         '__format__',
         '__ge__',
         '__getattribute__',
         '__getitem__',
         '__getnewargs__',
         '__gt__',
         '__hash__',
         '__init__',
         '__init_subclass__',
         '__iter__',
         '__le__',
         '__len__',
         '__lt__',
         '__mod__',
         '__mul__',
         '__ne__',
         '__new__',
         '__reduce__',
         '__reduce_ex__',
         '__repr__',
         '__rmod__',
         '__rmul__',
         '__setattr__',
         '__sizeof__',
         '__str__',
         '__subclasshook__',
         'capitalize',
         'casefold',
         'center',
         'count',
         'encode',
         'endswith',
         'expandtabs',
```

I don’t know all these methods

אוניברסיטת אריאל בשומרון

```
In [7]: help(text.upper)
```

```
Help on built-in function upper:

upper() method of builtins.str instance
    Return a copy of the string converted to uppercase.
```

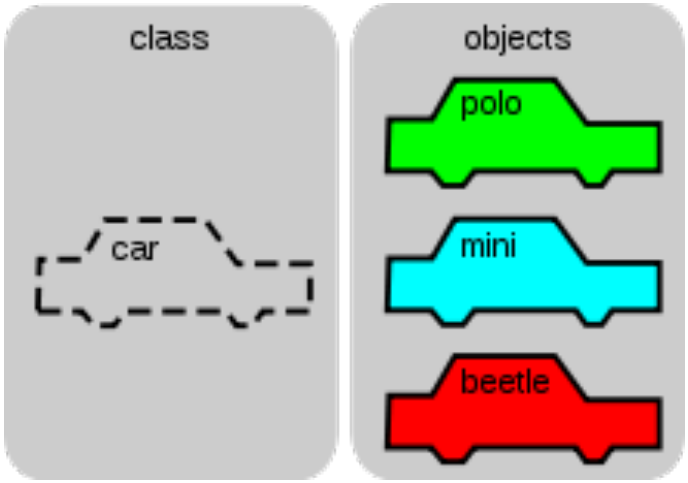
```
In [8]: help(number.to_bytes)
```

```
Help on built-in function to_bytes:

to_bytes(length, byteorder, *, signed=False) method of builtins.int instance
    Return an array of bytes representing an integer.

    length
      Length of bytes object to use.  An OverflowError is raised if the
      integer is not representable with the given number of bytes.
    byteorder
      The byte order used to represent the integer.  If byteorder is 'big',
      the most significant byte is at the beginning of the byte array.  If
      byteorder is 'little', the most significant byte is at the end of the
      byte array.  To request the native byte order of the host system, use
      `sys.byteorder' as the byte order value.
    signed
      Determines whether two's complement is used to represent the integer.
      If signed is False and a negative integer is given, an OverflowError
      is raised.
```

Classes as Data Types

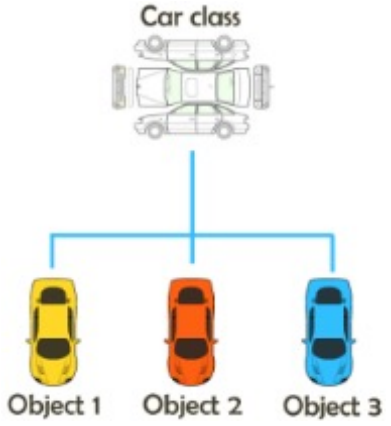
Classes define types that are:

- a **composition** of other types
- have unique **functionality**

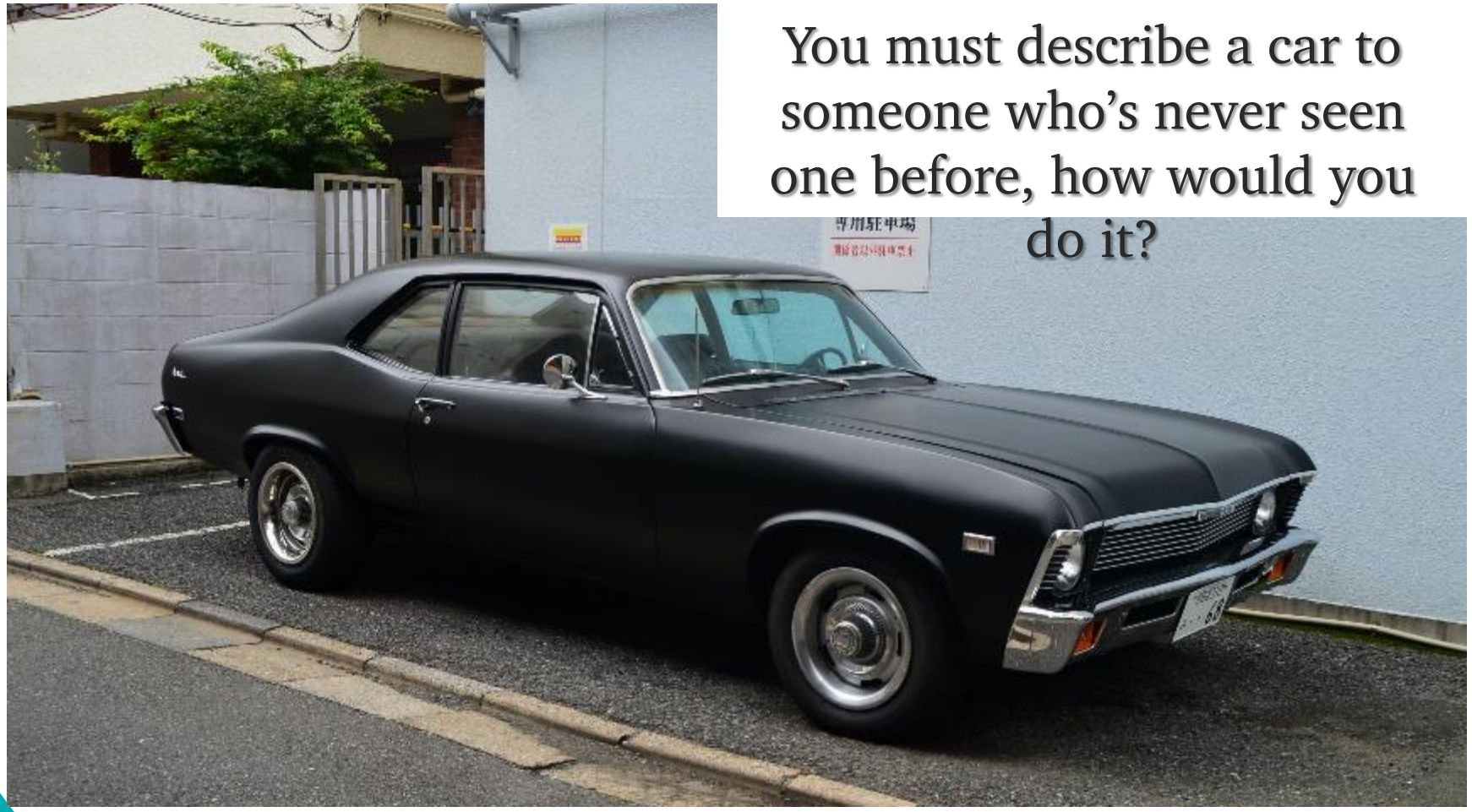


Every class may contain:

- Attributes (fields)
- Methods
- Constructor (Initialization function)



Car example



You must describe a car to someone who's never seen one before, how would you do it?

Car Example

אוניברסיטת אריאל בשומרון

Attributes 4 wheels, steering wheel, horn, color etc.

- Unique for every car instance

Methods drive, turn left, honk, repaint etc.

Constructor by color, by engine type etc..

Classes in Python

אוניברסיטת אריאל בשומרון

- Class definition looks like this:

```
class ClassName:
    """documentation string"""

    def __init__(self):
        # constructor

    def method_name(self, arg1, arg2):
        # method code

    def method_name(self, arg1, arg2):
        # method code

    ...
```

How to Represent a Point in 2D?

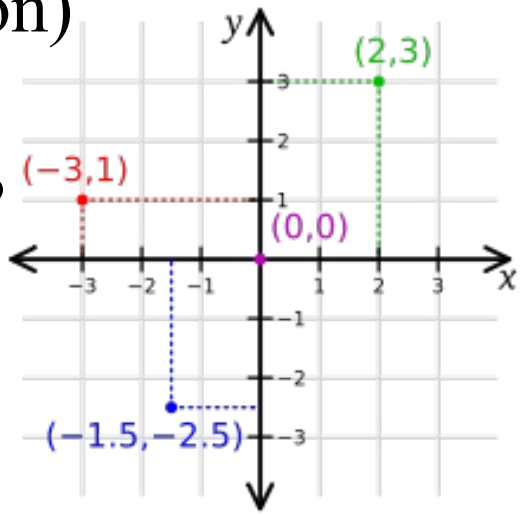
Alternatives

- Two variables x, y
- Elements in a list / tuple
- A new data type

Creating a new type is a (little) more complicated,

But has advantages (to be apparent soon)

So - How should we represent a point?



Creating a new Point

אוניברסיטת
אריאל
בשומרון

```
class Point:
    """represents a point is 2D space.
       Attributes: x, y"""
```

In the shell:

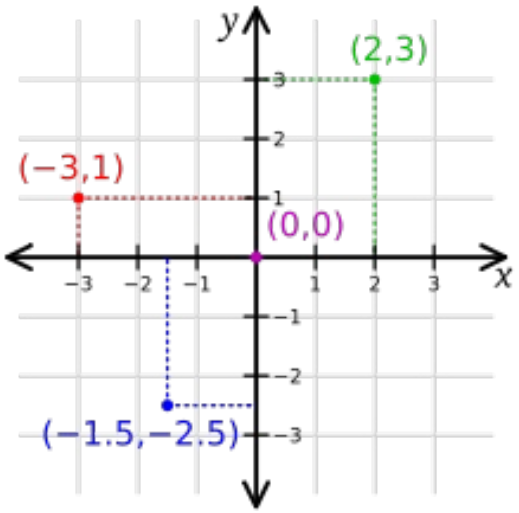
```
>>> blank = Point()
```

But where is the Point?

אוניברסיטת אריאל בשומרון

```
>>> blank = Point()
>>> blank.x

Traceback (most recent call last):
  File "<pyshell#8>", line 1, in <module>
    blank.x
AttributeError: 'Point' object has no attribute 'x'
```

- Currently, blank does not hold any data, so x does not exist (nor does y)
- We want to be able to **initialize** and **access** *x, y* via Point instance



Object initialization

אוניברסיטת
אריאל
בשומרון

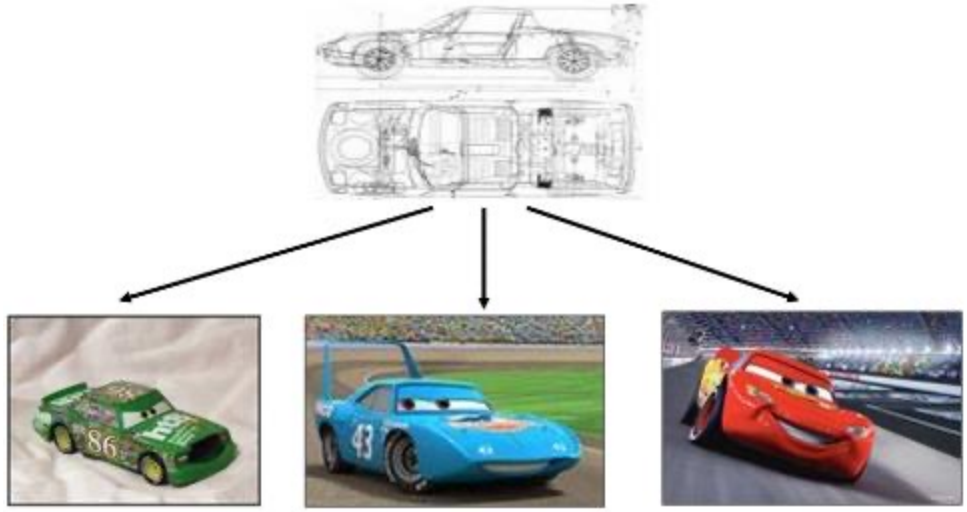
- There are two ways to initialize an object:

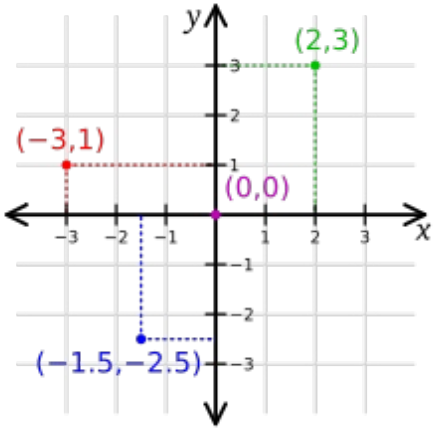
  1. Explicitly

```
>>> blank = Point()
>>> blank.x = 3.0
>>> blank.y = 4.0
>>> blank.x
3.0
>>> blank.y
4.0
```

  2. Using a **constructor**, which is a special function in charge of initializing the object's variables

Constructors





- Definition in the class' code that "produces" instances
- Invoked automatically when an object is created
- Input: whatever is required to produce a new instance

- What would a Point constructor accept as input?

Constructors – More Technically

אוניברסיטת אריאל בשומרון

```
class Point:
    """represents a point is 2D space.
        Attributes: x, y"""
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

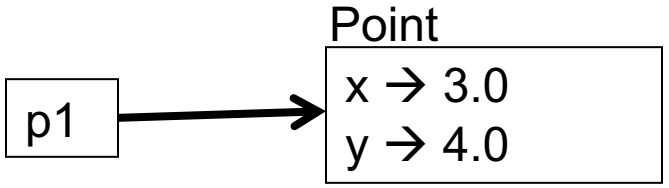
Definition within the class's scope:

- Always named *__init__*
  - 2 underscores, followed by the word init, followed by 2 more underscores.
- The first parameter is ***self***
  - A *reference* to the current instance of the class.
  - When calling the constructor, ***self*** is not passed, it is created by Python.
- Default constructor - If a constructor was not implemented for a class, Python assumes that the class has a default empty constructor (no arguments, no attribute initialization)

Attributes

אוניברסיטת
אריאל
בשומרון

```
>>> p1 = Point(3.0, 4.0)
>>> p1.x
3.0
>>> p1.y
4.0
```



- The variable **`p1`** refers to a Point object
- **`p1.x`** means "get the value of *x* from object p1"
- There is no conflict between a variable *x* and the attribute *x*

```
>>> x = p1.y
>>> print (x)
4.0
>>> print (p1.x)
3.0
```

Instances and Functions

אוניברסיטת אריאל בשומרון

Objects can be passed as arguments to functions:

```
def print_point(point):
    print('<', point.x, ',', point.y, '>')

>>> print_point(p1)
< 3.0 , 4.0 >
```

Objects are mutable:

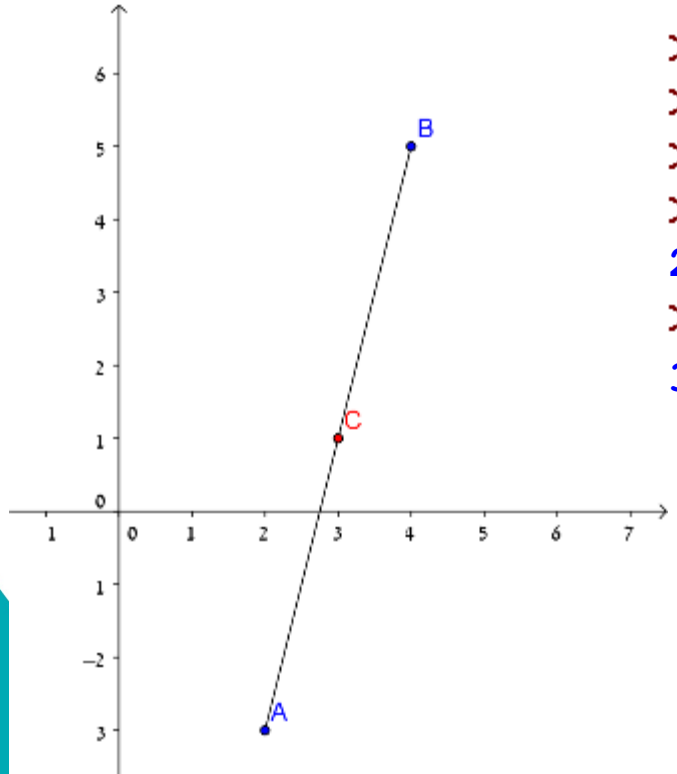
```
def move_point(point, dx, dy):
    point.x += dx
    point.y += dy

>>> move_point(p1, 2, -1)
>>> print_point(p1)
< 5.0 , 3.0 >
```

Instances and Functions

אוניברסיטת
אריאל
בשומרון

Objects can be returned by functions:

```
def middle_point(p1, p2):
    return Point((p1.x + p2.x)/2.0 , (p1.y + p2.y)/ 2.0)
```




```
>>> p1 = Point(1,1)
>>> p2 = Point(3,5)
>>> p3 = middle_point(p1, p2)
>>> p3.x
2.0
>>> p3.y
3.0
```

Object Oriented Programming

אוניברסיטת אריאל בשומרון

Programs are made of

- object definitions
- methods definitions

Most of the computation is in operations on objects

An object definition corresponds to objects or concepts in the real world.

The methods that operate on that objects correspond to the ways real-world objects interact.

Methods

אוניברסיטת אריאל בשומרון

Method

a function that is associated with a particular class.

Examples in strings, lists, dictionaries, tuples:

list.append(), str.upper(), dict.items()

Difference between methods and functions:

- Methods are defined inside a class definition
- The syntax for invoking a method is different:
  - A method is called through an instance

Methods

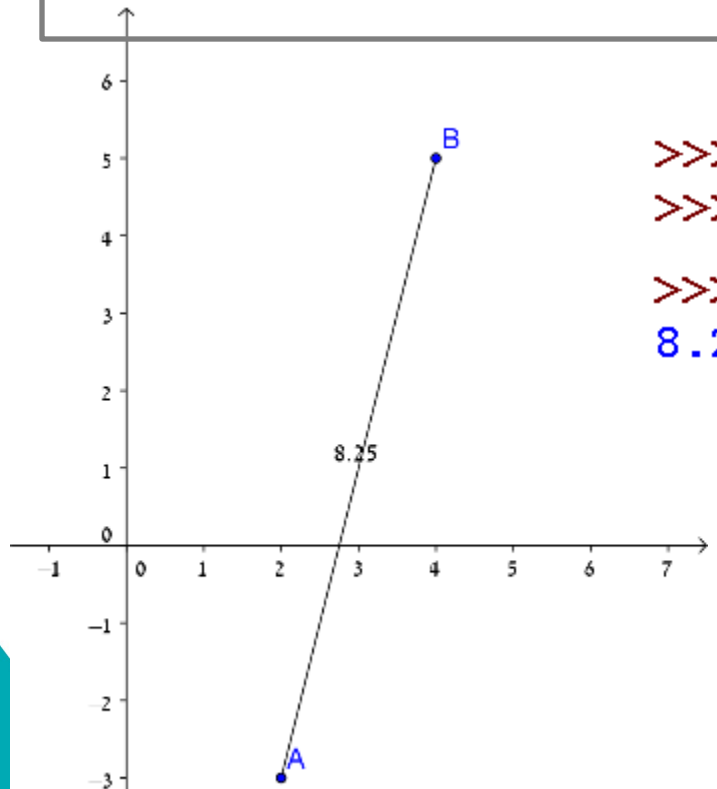
1. The first argument of each method is **self**. It's not passed as an argument in the call – **self** is the object that invoked the method.
2. Access the attributes of the class – only with self.

```
class New_Class:
        """documentation string"""

        def __init__(self, att1):
            self.att1 = att1;

        def method_name(self, arg1, arg2):
              do_something_with(self.att1)
```

Example: Distance Between Two Points (method)

אוניברסיטת אריאל בשומרון

```
from math import sqrt
    def distance(self, other):
        return sqrt((self.x-other.x)**2 + (self.y-other.y)**2)
```




```
>>> p1 = Point(2,-3)
>>> p2 = Point(4,5)

>>> p1.distance(p2)
8.246211251235321
```

Example: Distance Between Two Points (method vs. function)

אוניברסיטת אריאל בשומרון

```
from math import sqrt

class Point:
    .......
    def distance(self, other):
        return sqrt((self.x-other.x)**2 + (self.y-other.y)**2)
    .......

def distance(p1, p2):
    return sqrt((p1.x - p2.x)**2 + (p1.y - p2.y)**2)
```

```
>>> p1 = Point(2,-3)
>>> p2 = Point(4,5)
>>> p1.distance(p2)
8.246211251235321
>>> distance(p1,p2)
8.246211251235321
```

Method call

function call

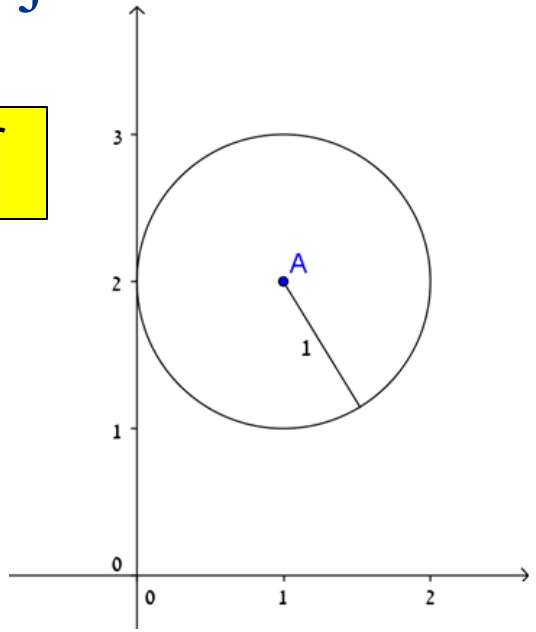
Let’s work – Q1

- Create 4 points, each with a random value for the x, y fields
- For each point, find the closest point, and print their distance

A Circle

- How would you represent a circle object?
- Attributes:
  - center (Point)
  - radius (int/float)

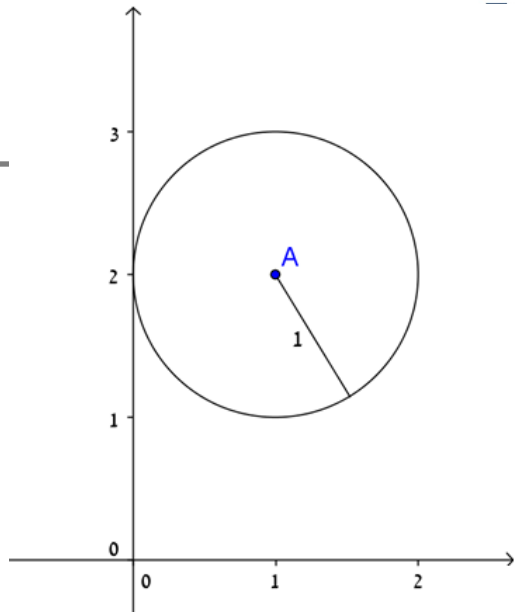
Composition of other user defined types



- Methods:

Method name	description	arguments	return value
__init__	The constructor	Center(Point), radius(int/float)	-
print_circle	Prints circle	-	-
in_circle	Checks if a point is located inside the circle	a *point* object	True/False

A Circle




```
class Circle:
    """ represents a circle shape
    attributes: center, radius"""
    def __init__(self, center, radius):
        self.center = center
        self.radius = radius

    def print_circle(self):
        print('center:', self.center.x,self.center.y,
              ', radius:', self.radius)
```

Next week – make str(center) work!

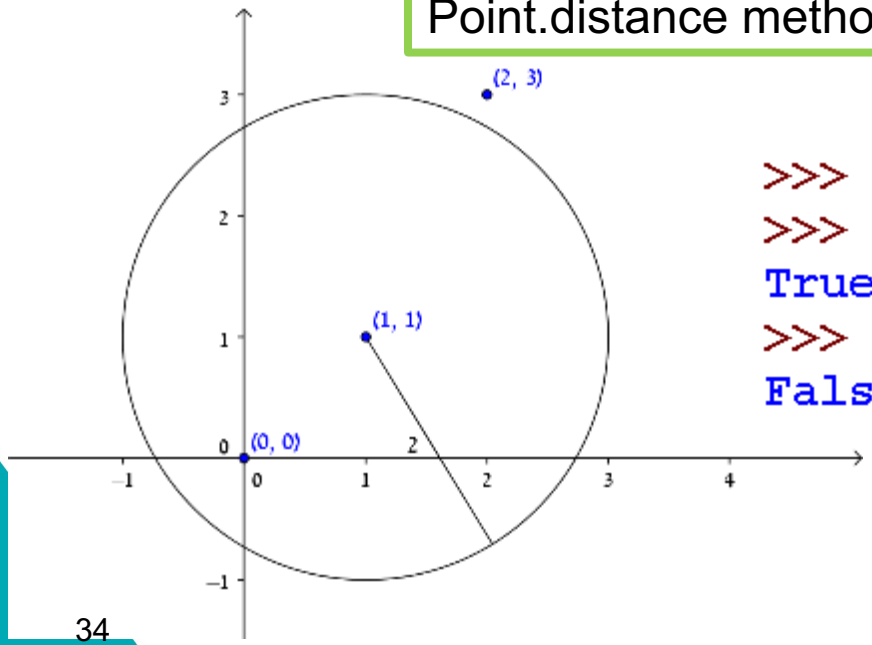
```
>>> c = Circle(Point(1,2),1)
>>> c.print_circle()
center: 1 2 , radius: 1
```

Example: In Circle

```
class Circle:
    .....
    def in_circle(self, point):
        return self.center.distance(point) < self.radius
```

Point.distance method call

Boolean value




```
>>> c = Circle(Point(1,1), 2)
>>> c.in_circle(Point(0,0))
True
>>> c.in_circle(Point(2,3))
False
```

A Rectangle (design options)

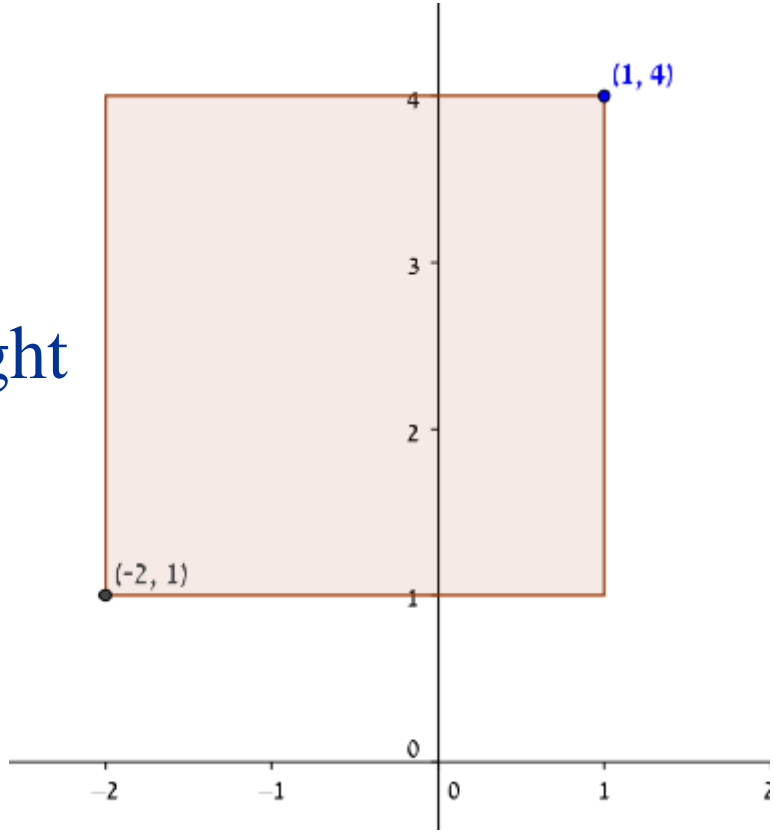
אוניברסיטת אריאל בשומרון

How would you represent a rectangle?

For simplicity ignore angle, assume the rectangle is vertical or horizontal

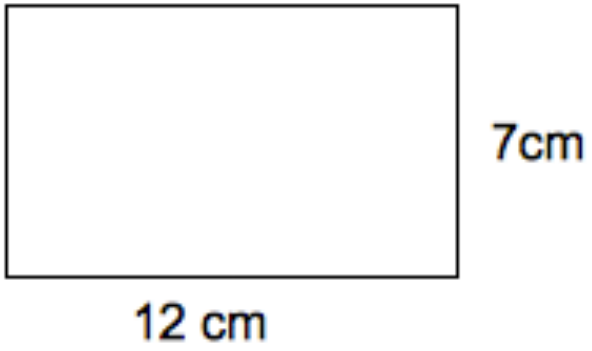
Several possibilities:

- Two opposing corners
- One corner + width and height



Rectangle - Design

- Attributes:
  - width (int/float)
  - height (int/float)
  - corner (Point)



- Methods

Method name	description	arguments	return value
__init__	The constructor	width, height, corner	-
print_rec	Prints the rectangle	-	-
get_center	Returns the center of the rectangle	-	A *Point* object that represents the center.

A Rectangle - Implementation

```
class Rectangle:
    """represents a rectangle in a 2D space.
    attributes: width, height, lower-left corner"""
    def __init__(self, width, height, corner):
        self.width = width
        self.height = height
        self.corner = corner

    def print_rec(self):
        print('Width:', self.width, ', Height:', \
              self.height, ', Lower-left corner:',\
              self.corner.x, self.corner.y)
```

```
>>> rect = Rectangle(100, 200, Point(0,0))
>>> rect.print_rec()
Width: 100 , Height: 200 , Lower-left corner: 0 0
```

Find Center

```
class Rectangle:

    def get_center(self):
        center_x = self.corner.x + self.width/2.
        center_y = self.corner.y + self.height/2.
        return Point(center_x, center_y)
```

```
>>> center = rect.get_center()
>>> center.print_point()
< 50.0 , 100.0 >
```